# 零基础读懂 ch03.ipynb

从「拿一句话喂给注意力」到「多头注意力」

2026 年 4 月 21 日

## 目录


**1 这一章在解决什么** **3**

**2 讲解原则** **3**

**3 零基础最该问的 7 个问题** **3**

**4 贯穿全章的玩具例子** **4**

**5 第一步：没有任何参数的注意力（§3.3）** **4**

5.1 背景问题 . . . . . 4

5.2 朴素直觉 . . . . . 5

5.3 数学表达 . . . . . 5

5.4 代码对应 . . . . . 5

5.5 把 6 个查询者一次算完 . . . . . 6

**6 第二步：把可训练参数加进去（§3.4）** **6**

6.1 背景问题 . . . . . 6

6.2 朴素直觉：把每个词分裂成三个角色 . . . . . 6

6.3 数学表达：缩放点积注意力 . . . . . 6

6.4 代码对应：`SelfAttention_v1` 与 `_v2` . . . . . 7

**7 第三步：让模型「不许偷看未来」（§3.5）** **8**

7.1 背景问题 . . . . . 8

7.2 朴素直觉：把右上角抹掉 . . . . . 8

7.3 两种实现方式 . . . . . 8

7.4 代码对应 . . . . . 8

7.5 再加一层 dropout（§3.5.2） . . . . . 9

7.6 封装成 `CausalAttention`（§3.5.3） . . . . . 9

**8 第四步：从一个头到多头（§3.6） 10**

8.1 背景问题 . . . . . 10

8.2 教学版：把多个 `CausalAttention` 摞起来（§3.6.1） . . . . . 10

8.3 生产版：权重切分 + 一次矩阵乘（§3.6.2） . . . . . 10

**9 把整章压成一张表 12**

**10 这一节真正该记住的 12**

# 1 这一章在解决什么

**问题**

GPT 一类的语言模型在做一件事：给定前面已经写好的若干个词，预测下一个词是什么。要让模型「猜得准」，每个位置都需要参考它前面所有的词。最早的做法是用循环网络（RNN）一个一个 token 处理，最后挤出一个固定大小的隐藏状态。当句子变长，这个隐藏状态就变成了瓶颈：远处的信息要么被压扁，要么被忘掉。

**注意力机制（Attention）就是为了解决这个瓶颈而生：**让每个位置可以直接看其它每个位置，根据「相关程度」加权，再把信息汇总过来。

第 3 章干的事，就是把「注意力」从最简单的雏形，一步一步搭成 GPT 真正使用的那个版本。

**朴素直觉**

你可以把一句话想成 6 个人坐成一排开会。每个人都有一句话要说（输入向量），但他在最终发言之前，会先扫一眼屋里所有人，根据「这个人对我有多重要」打一个权重，再把大家的话按权重加起来，作为他真正的「上下文版本」。注意力机制就是把这个「扫一眼 + 加权汇总」用数学和代码写出来。

**记住这一点**

本章的最终交付物是一个叫 `MultiHeadAttention` 的 PyTorch 模块。后面所有章节的 GPT 都直接拿它当积木。

# 2 讲解原则

为了不一上来就被符号淹没，本讲义对每一节都按下面四步展开：

1. **背景问题**：上一节做完了，新的麻烦是什么？
2. **朴素直觉**：先用人话说清楚，不引入新符号。
3. **数学表达**：把直觉写成公式，每个新符号都立刻翻译成中文。
4. **代码对应**：在 notebook 里这一段长什么样、张量形状是什么。

如果某一节让你觉得「公式比直觉先到」，回到第 2 步。

# 3 零基础最该问的 7 个问题

**Q1.** 什么叫「一个词的向量表示」？为什么句子可以变成一堆数？

**Q2.**「注意力分数」到底在量什么？为什么是点积？

**Q3.** 为什么要 softmax？不能直接归一化吗？

**Q4.** Query、Key、Value 这三个词差在哪？为什么要分三份？

**Q5.** 为什么后面要除以 $\sqrt{d_k}$？

**Q6.**「因果掩码（causal mask）」是干嘛的？为什么用 $-\infty$？

**Q7.**「多头」到底多在哪？是把网络做大了吗？

下面所有节都在回答其中一个或几个问题。

# 4 贯穿全章的玩具例子

**问题**

全章都用同一个 6 个词的英文小句作为玩具：

$$\textit{Your} \quad \textit{journey} \quad \textit{starts} \quad \textit{with} \quad \textit{one} \quad \textit{step}$$

每个词被替换成一个 3 维向量（这一步在第 2 章里讲过，叫 *embedding*），拼起来就是一个形状 $(6, 3)$ 的矩阵 `inputs`。

**代码对应**

```
inputs = torch.tensor(
  [[0.43, 0.15, 0.89], # Your x^(1)
   [0.55, 0.87, 0.66], # journey x^(2)
   [0.57, 0.85, 0.64], # starts x^(3)
   [0.22, 0.58, 0.33], # with x^(4)
   [0.77, 0.25, 0.10], # one x^(5)
   [0.05, 0.80, 0.55]] # step x^(6)
)
```

**朴素直觉**

**这些数字是什么意思？**

不要纠结具体数值——它们只是「每个词在 3 个隐含维度上的强度」。真实模型里这些维度是几百到几千，并且是训练学出来的。本章不学怎么得到这些数，本章学的是：已经有了这些数之后怎么用注意力机制把它们重新汇总。

之后凡是要看「中间结果长什么样」，几乎都用第 2 个词 *journey*（也就是 `inputs[1]`）当主角。

# 5 第一步：没有任何参数的注意力（§3.3）

## 5.1 背景问题

我们想给每个词产出一个新的向量 $z^{(i)}$，叫做**上下文向量（context vector）**。它必须同时融合所有 6 个词的信息，但不同词的贡献要不一样。最朴素的问题是：「贡献的权重从哪来？」

## 5.2 朴素直觉

**朴素直觉**

能不能直接用「两个词的相似度」当权重?
两个向量越像，它们的点积就越大。所以一个最朴素的想法是：

1. 拿当前这个词 $\boldsymbol{x}^{(i)}$ 跟所有 6 个词分别做点积，得到 6 个分数；
2. 把 6 个分数压成「和为 1」的权重；
3. 用权重把 6 个原始向量加起来。

整个过程没有任何要学习的参数，纯计算。

## 5.3 数学表达

以 $\boldsymbol{x}^{(2)}$ 为「查询者」为例。

**第一步：注意力分数（attention scores）**

$$\omega_{2i} \;=\; \boldsymbol{x}^{(i)} \cdot \boldsymbol{x}^{(2)}, \qquad i = 1, 2, \ldots, 6.$$

每个 $\omega_{2i}$ 是一个数：第 $i$ 个词跟第 2 个词有多「像」。

**第二步：注意力权重（attention weights）**

$$\alpha_{2i} \;=\; \frac{\exp(\omega_{2i})}{\sum_{j=1}^{6} \exp(\omega_{2j})}.$$

这就是 **softmax**：把任意实数压成「全为正、加起来等于 1」的概率。

**备注 5.1.** 为什么不能直接做 $\alpha_{2i} = \omega_{2i} / \sum_j \omega_{2j}$？因为分数可能是负数，相除以后会出现「负权重」，失去「权重」的语义。*softmax* 先用 exp 把所有数变正，再归一化，所以无论原始分数多大多小，结果都是合法的概率分布。

**第三步：加权求和**

$$\boldsymbol{z}^{(2)} \;=\; \sum_{i=1}^{6} \alpha_{2i}\, \boldsymbol{x}^{(i)}.$$

$\boldsymbol{z}^{(2)}$ 是一个新的 3 维向量，它就是「以 $\boldsymbol{x}^{(2)}$ 为视角看到的整句话」。

## 5.4 代码对应

**代码对应**

```
query = inputs[1] # x^(2)
attn_scores_2 = inputs @ query # 6 个分数
attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
context_vec_2 = attn_weights_2 @ inputs # 形状 (3,)
# -> tensor([0.4419, 0.6515, 0.5683])
```

### 5.5 把 6 个查询者一次算完

把上面对单个 $\boldsymbol{x}^{(2)}$ 做的事，对 6 个词同时做。让 $X$ 表示 $(6,3)$ 的输入矩阵，则

$$\Omega = XX^\top \in \mathbb{R}^{6\times 6}, \qquad A = \mathrm{softmax}_{\text{行}}(\Omega), \qquad Z = AX \in \mathbb{R}^{6\times 3}.$$

**代码对应**

```
attn_scores = inputs @ inputs.T # (6, 6)
attn_weights = torch.softmax(attn_scores, dim=-1) # 每行 sum=1
all_context_vecs = attn_weights @ inputs # (6, 3)
```

**记住这一点**

注意力 =「拿点积当相似度，softmax 当权重，加权求和当输出」。本节没有任何可学习参数，只是把这套套路写清楚。

## 6 第二步：把可训练参数加进去（§3.4）

### 6.1 背景问题

§3.3 的版本有个尴尬：「相似度」完全由原始词向量决定，模型没有任何「调节空间」。真实任务里我们希望模型自己学到：「在做这个任务时，词与词之间到底应该看哪些维度才算有关？」

### 6.2 朴素直觉：把每个词分裂成三个角色

**朴素直觉**

还是那个开会的比喻。每个人现在被要求扮演三个角色：

- **Query（查询者）**：「我想问什么？」
- **Key（钥匙）**：「关于我的信息，可以用什么标签找到我？」
- **Value（值）**：「如果你真的看了我，我会贡献给你什么？」

判断「该不该听某人」用 *query* 和 *key* 做点积。真正搬过来的内容是 *value*。而把同一个原始向量分裂成这三个版本的，是三个可训练的矩阵 $W_q, W_k, W_v$。

### 6.3 数学表达：缩放点积注意力

设输入维度 $d_{\text{in}}$，输出维度 $d_{\text{out}}$，三个投影矩阵

$$W_q, W_k, W_v \in \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}.$$

对每个词 $\boldsymbol{x}^{(i)}$ 算出三个版本：

$$\boldsymbol{q}^{(i)} = \boldsymbol{x}^{(i)}W_q, \quad \boldsymbol{k}^{(i)} = \boldsymbol{x}^{(i)}W_k, \quad \boldsymbol{v}^{(i)} = \boldsymbol{x}^{(i)}W_v.$$

注意力分数和权重写成

$$\alpha_{ij} \;=\; \mathrm{softmax}_j\left(\frac{\boldsymbol{q}^{(i)}\cdot\boldsymbol{k}^{(j)}}{\sqrt{d_k}}\right), \qquad d_k = d_{\mathrm{out}}.$$

最终

$$\boldsymbol{z}^{(i)} \;=\; \sum_j \alpha_{ij}\,\boldsymbol{v}^{(j)}.$$

矩阵形式（这就是 transformer 论文那个出名的式子）：

$$\boxed{\mathrm{Attention}(Q,K,V) \;=\; \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.}$$

### 为什么除以 $\sqrt{d_k}$？

**朴素直觉**

$\boldsymbol{q}\cdot\boldsymbol{k}$ 是 $d_k$ 项相加。维度 $d_k$ 越大，点积的数值波动也越大，softmax 容易把几乎全部概率塞给单独一个位置，变成「非黑即白」的硬选择，梯度会几乎消失。除以 $\sqrt{d_k}$ 把方差拉回正常范围，softmax 才有「软选择」的余地，模型才训得动。

## 6.4 代码对应：`SelfAttention_v1` 与 `_v2`

notebook 里把上面这套封装成两个 `nn.Module` 类，结构完全一致，区别只在「投影矩阵怎么写」：

**代码对应**

```python
class SelfAttention_v1(nn.Module): # 用裸的 nn.Parameter
    def __init__(self, d_in, d_out):
        super().__init__()
        self.W_query = nn.Parameter(torch.rand(d_in, d_out))
        self.W_key = nn.Parameter(torch.rand(d_in, d_out))
        self.W_value = nn.Parameter(torch.rand(d_in, d_out))
    def forward(self, x):
        Q, K, V = x @ self.W_query, x @ self.W_key, x @ self.W_value
        scores = Q @ K.T
        weights = torch.softmax(scores / K.shape[-1] ** 0.5, dim=-1)
        return weights @ V

class SelfAttention_v2(nn.Module): # 用更标准的 nn.Linear
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
    def forward(self, x):
        Q, K, V = self.W_query(x), self.W_key(x), self.W_value(x)
        scores = Q @ K.T
```

```
    weights = torch.softmax(scores / K.shape[-1] ** 0.5, dim=-1)
    return weights @ V
```

**备注 6.1.** *v1 和 v2 输出的数不一样，不是因为算法不同，而是因为 nn.Linear 的默认初始化方式比 torch.rand 好得多。真实代码里都用 v2。*

# 7 第三步：让模型「不许偷看未来」(§3.5)

## 7.1 背景问题

GPT 是自回归的：在预测第 $i$ 个词时，它只能看 $1, 2, \ldots, i$ 这些位置，**绝对不能看到 $i$ 以后的词**——因为训练时那些词正是它要预测的答案。但 §3.4 的注意力对 6 个位置一视同仁，没有任何方向感。我们要给它加「方向」。

## 7.2 朴素直觉：把右上角抹掉

**朴素直觉**

注意力权重是一个 $(6, 6)$ 的方阵：第 $i$ 行表示「位置 $i$ 在看哪些位置」。要让它「不许向右看」，只需保证第 $i$ 行里第 $i+1, i+2, \ldots$ 列的权重都是 0。也就是把整个矩阵的右上三角抹成 0，剩下一个下三角。

## 7.3 两种实现方式

**做法 A（教学版）**：先 softmax 再乘下三角 mask，最后重新归一化让每行和等于 1。能跑，但要做两次归一化，$\boxed{\text{F}}$嗦。

**做法 B（标准版）**：在 softmax 之前，把上三角的分数直接设成 $-\infty$。

**为什么 $-\infty$ 这么神奇？**

softmax 第一步是 $\exp(\cdot)$，而 $\exp(-\infty) = 0$。所以「-∞ 的分数」会自动得到「0 的权重」，剩下的位置自然加起来等于 1，只要做一次 softmax 就完成了「屏蔽 + 归一化」。

## 7.4 代码对应

**代码对应**

```
mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
# mask 是上三角全 1、对角及以下全 0
masked_scores = scores.masked_fill(mask.bool(), -torch.inf)
weights = torch.softmax(masked_scores / d_k ** 0.5, dim=-1)
```

### 7.5 再加一层 dropout（§3.5.2）

**朴素直觉**

**Dropout** 是常见的正则化：训练时随机把一部分数值置 0，让模型不要过度依赖任何单一连接。这里把 dropout 直接作用在注意力权重矩阵上：随机丢弃一些「我在看你」的连线。
为了让被保留下来的位置「期望值不变」，PyTorch 会把保留的元素乘 $\frac{1}{1-p}$（$p$ 是 dropout 比率）。例如 $p = 0.5$ 时，留下来的元素都 $\times 2$。

### 7.6 封装成 `CausalAttention`（§3.5.3）

到这里我们把「可训练 + 因果 + dropout + 支持 batch」打包成一个干净的类：

**代码对应**

```python
class CausalAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, qkv_bias=False):
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1)
        )

    def forward(self, x): # x: (b, T, d_in)
        b, T, _ = x.shape
        Q, K, V = self.W_query(x), self.W_key(x), self.W_value(x)
        scores = Q @ K.transpose(1, 2) # (b, T, T)
        scores.masked_fill_(self.mask.bool()[:T, :T], -torch.inf)
        weights = torch.softmax(scores / K.shape[-1] ** 0.5, dim=-1)
        weights = self.dropout(weights)
        return weights @ V # (b, T, d_out)
```

三个值得注意的小细节：

- `register_buffer`：mask 不是要训练的参数，但要跟着模型一起搬到 GPU，所以注册为 buffer。
- `transpose(1, 2)`：现在多了一个 batch 维，所以转置的是后两维。
- 切片 `[:T, :T]`：mask 是按最大可能长度建的，实际输入比较短时切一块出来用。

# 8 第四步：从一个头到多头（§3.6）

## 8.1 背景问题

一个注意力头只能学一种「关注模式」。但语言里同时存在很多种关系：词法依赖、指代、修饰、远距离呼应……一个头很难全管。**多头注意力**的想法是：让若干个头并行地学不同的关注模式，最后把它们的结果拼起来。

## 8.2 教学版：把多个 CausalAttention 摞起来（§3.6.1）

**代码对应**

```
class MultiHeadAttentionWrapper(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False
        ):
        super().__init__()
        self.heads = nn.ModuleList([
            CausalAttention(d_in, d_out, context_length, dropout, qkv_bias)
            for _ in range(num_heads)
        ])
    def forward(self, x):
        return torch.cat([h(x) for h in self.heads], dim=-1)
```

**朴素直觉**

**它清楚但慢。**

$N$ 个头要跑 $N$ 次循环，硬件白白闲着。下面这个生产版会把 $N$ 个头压进一次矩阵乘法。

## 8.3 生产版：权重切分 + 一次矩阵乘（§3.6.2）

核心想法：

1. 还是用一个大的 `nn.Linear(d_in, d_out)` 做 $Q, K, V$，但要求 `d_out` 已经能被 `num_heads` 整除。
2. 把最后一维 `d_out` 重塑成 `(num_heads, head_dim)`。
3. 转置让 `num_heads` 跑到前面，作为新的「批量轴」。
4. 这样「每个头各自做缩放点积注意力」就变成一次带 batch 的矩阵乘法。
5. 算完再转置回去、压平最后两维，最后过一个输出投影 `out_proj`。

**形状变换**

$$\underbrace{(b, T, d_{\text{out}})}_{\text{投影后}} \xrightarrow{\text{view}} (b, T, h, d_h) \xrightarrow{\text{transpose}(1,2)} (b, h, T, d_h),$$

其中 $h$ 是头数，$d_h = d_{\text{out}}/h$。

**代码对应**

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False
        ):
        super().__init__()
        assert d_out % num_heads == 0
        self.d_out, self.num_heads = d_out, num_heads
        self.head_dim = d_out // num_heads
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1)
        )

    def forward(self, x): # x: (b, T, d_in)
        b, T, _ = x.shape
        Q = self.W_query(x).view(b, T, self.num_heads, self.head_dim).transpose(1, 2)
        K = self.W_key(x) .view(b, T, self.num_heads, self.head_dim).transpose(1, 2)
        V = self.W_value(x).view(b, T, self.num_heads, self.head_dim).transpose(1, 2)
        # 现在 Q, K, V 形状都是 (b, num_heads, T, head_dim)

        scores = Q @ K.transpose(2, 3) # (b, h, T, T)
        scores.masked_fill_(self.mask.bool()[:T, :T], -torch.inf)
        weights = torch.softmax(scores / K.shape[-1] ** 0.5, dim=-1)
        weights = self.dropout(weights)

        ctx = (weights @ V).transpose(1, 2) # (b, T, h, d_h)
        ctx = ctx.contiguous().view(b, T, self.d_out) # 拼回 d_out
        return self.out_proj(ctx) # (b, T, d_out)
```

**备注 8.1.** 注意 *MultiHeadAttention* 和 *MultiHeadAttentionWrapper* 的输出维度约定不同：前者的 *d_out* 已经是「所有头加起来」的总维度，后者是「每个头独立的 *d_out*，再拼起来」。所以同一组参数下，前者输出 *(2, 6, 2)*，后者输出 *(2, 6, 4)*。

# 9 把整章压成一张表

| 小节 | 写法 | 可训练 | 因果 | dropout | 多头 |
|---|---|---|---|---|---|
| §3.3.1 | 手写循环 / 点积 | 否 | 否 | 否 | 1 |
| §3.3.2 | `inputs @ inputs.T` | 否 | 否 | 否 | 1 |
| §3.4.1 | 手写 $W_q, W_k, W_v$ | 是 | 否 | 否 | 1 |
| §3.4.2 | `SelfAttention_v1/v2` | 是 | 否 | 否 | 1 |
| §3.5.1 | + 因果 mask（$-\infty$） | 是 | 是 | 否 | 1 |
| §3.5.2 | + dropout | 是 | 是 | 是 | 1 |
| §3.5.3 | `CausalAttention`（带 batch） | 是 | 是 | 是 | 1 |
| §3.6.1 | `MultiHeadAttentionWrapper` | 是 | 是 | 是 | N（堆叠） |
| §3.6.2 | `MultiHeadAttention` | 是 | 是 | 是 | N（并行） |

# 10 这一节真正该记住的

1. 注意力的本质是**加权汇总**：权重来自 $\mathrm{softmax}(QK^\top/\sqrt{d_k})$，汇总的对象是 $V$。
2. $W_q, W_k, W_v$ 是**可学习参数**，让模型能为不同任务挑出该看的维度。
3. $\sqrt{d_k}$ 是**为了不让 softmax 极化**，否则梯度会消失。
4. 因果掩码用 $-\infty$ 而不是 0，是因为它要在 softmax 之前生效，利用 $\exp(-\infty) = 0$ 自动屏蔽未来。
5. dropout 用在**注意力权重矩阵上**，是为了别让模型死记某条「位置 $i \to j$」的连线。
6. 多头注意力 = 把 $d_{\text{out}}$ 拆成「头数 × 每头维度」，然后让头充当批量轴，一次矩阵乘把所有头算完，最后再过一次 `out_proj` 把它们融合起来。
7. `MultiHeadAttention` 这个类，就是第 4 章 GPT 模型里的注意力积木。

**记住这一点**

读完这一章，你应该能在白板上不查代码地写出：
1) 缩放点积注意力的公式；
2) 因果掩码为什么用 $-\infty$；
3) 多头注意力怎么用「reshape + transpose」一次算完。